



Bulut Bilişim Ve Uygulamaları

BLM(3522)

OTOMATİK ÖLÇEKLENDİRME VE YÜK
DENGELEME TABANLI E-TİCARET MİMARİSİ -
PROJE GELİŞTİRME DOKÜMANTASYONU

Hazırlayan: Berkay Bozlar

Numara: 23290939

Fakülte: Mühendislik Fakültesi

Bölüm: Bilgisayar Mühendisliği

GitHub:(<https://github.com/BerkayBozlar/BulutEticaret>)

Tanıtım Videosu:

(https://drive.google.com/file/d/1iwk3cl31A230qD2t_ZJ7_32UCRAIzToE/view?usp=drive_link)

Projenin temel amacı, modern e-ticaret sistemlerinin "Efsane Cuma" (Black Friday) gibi yoğun kampanya dönemlerinde maruz kaldığı ani trafik artışlarını yönetebilmek için AWS bulut altyapısı üzerinde dinamik, kendini onarabilen ve maliyet odaklı bir Auto Scaling (Otomatik Ölçeklendirme) mimarisi kurmaktır.

Bu doğrultuda, sunucu işlemcisini kasten yoran matematiksel bir Python (Flask) web uygulaması yazılmış, gelen HTTP ağ trafiği Application Load Balancer (ALB) ile dağıtılmış ve Amazon CloudWatch üzerinden CPU kullanımının %10 barajını aşması durumunda sistemin yatayda otomatik olarak yeni sunucular açması (Scale-Out), kriz bittiğinde ise eski haline dönmesi (Scale-In) uçtan uca simüle edilmiştir.

BÖLÜM 1: PROJENİN BAŞLATILMASI VE BULUT MİMARİSİ ALTYAPISININ KURULMASI

Sistemin stres testine tabi tutulmadan önce, bulut üzerinde kendi kendini kopyalayabilen ve dış dünyaya hizmet veren temel ağ mimarisi inşa edilmiştir.

1. Proje Geliştirme Ortamı

- **İşletim Sistemi & Ortam:** Amazon Linux 2023, AWS EC2 SSH Terminali
- **Programlama Dili & Framework:** Python 3.x, Bash Scripting, Flask
- **Kullanılan AWS Servisleri:** EC2, Auto Scaling Groups (ASG), Application Load Balancer (ALB), Amazon CloudWatch, Launch Templates.
- **Proje Adı:** BulutEticaret

2. EC2 Launch Template (Başlatma Şablonu) ve "User Data"

Otomasyonu Auto Scaling motorunun yeni bir sunucu açması gerektiğinde kullanacağı "kurabiye kalıbı" mimarisi oluşturulmuştur.

- **Donanım & İmaj:** t2.micro sunucu tipi ve Amazon Linux 2023 (AMI) seçilmiştir.
- **Sıfır Müdahale Kurulumu:** Şablonun *User Data* (Kullanıcı Verisi) bölümüne özel bir Bash betiği yazılarak, AWS'nin yeni açtığı her sunucuya insan müdahalesi olmadan otomatik olarak Python kurması, Flask web kodunu oluşturması ve uygulamayı Port 80 üzerinden yayına alması sağlanmıştır.

3. Güvenlik Duvarı (Security Group) ve Port Yönetimi

Bulut sunucularına yetkisiz erişimi engellemek için **Eticaret-SG** isimli özel bir güvenlik duvarı oluşturulmuştur.

- Uygulamanın dış dünyaya hizmet verebilmesi için **HTTP (Port 80)** erişimi "Anywhere (0.0.0.0/0)" olarak ayarlanarak tüm dünya trafiğine açılmıştır.
- Stres testleri ve sistem içi manuel müdahaleler (Sızma) yapılacağı anlarda **SSH (Port 22)** erişimi geçici olarak aktif edilmiş, iş bitiminde kapatılarak sistem güvenliği sağlanmıştır.

4. Application Load Balancer (Trafik Polisi) Entegrasyonu

Kullanıcıların doğrudan IP adresi üzerinden tek bir sunucuya yüklenmesini engellemek amacıyla Uygulama Yük Dengeleyici (ALB) devreye alınmıştır. ALB, gelen trafiği **Eticaret-TG** (Target Group) içerisinde bulunan sağlıklı (Healthy) sunuculara eşit şekilde (Round-Robin mantığıyla) dağıtmaktadır.

BÖLÜM 2: CORE BACKEND (E-TİCARET SİMÜLASYONU) VE OTOMASYON POLİTİKALARI

Bu aşamada, bulut mimarisinin ağır yük altında nasıl tepki verdiğini test edebilmek amacıyla simülatör uygulamasının ve CloudWatch ölçüm kurallarının kodlaması yapılmıştır.

1. Flask Web Sunucusu ve Kasıtlı İşlemci Yüğü (CPU Stressing) Gerçek bir Black Friday trafiğini simüle etmek için app.py isimli bir Python betiğı kodlanmıştır.

- Kullanıcı ana sayfadaki "Hemen Satın Al" butonuna tıkladığında, arka planda 1'den 15 milyona kadar karekök hesaplayan devasa bir for döngüsü tetiklenmektedir.
- Bu döngü, o anki sunucunun donanım kapasitesini anlık olarak zorlayarak CPU kullanımını tavan yaptırmak üzere özel olarak kurgulanmıştır.

2. Auto Scaling Group (ASG) ve Dinamik Ölçeklendirme Kuralı

Sunucuları gruplamak ve sayılarını yönetmek için **Eticaret-ASG** grubu kurulmuştur.

- **Kapasite Sınırları:** Minimum 1, İstenen (Desired) 1, Maksimum 3 sunucu olacak şekilde ayarlanmıştır.
- **Hedef İzleme (Target Tracking) Politikası:** Amazon CloudWatch üzerinden anlık izleme başlatılmış ve "Ortalama CPU kullanımı %10'u aşarsa otomatik olarak yeni sunucu aç" kuralı sisteme entegre edilmiştir.

BÖLÜM 3: SİSTEMİN CANLI TESTİ (STRES TESTİ) VE KENDİNİ ONARMA (SELF-HEALING)

Projenin son aşamasında bulut mimarisi zorlu bir siber trafik testine (Stress Test) tabi tutulmuş ve AWS'nin buna verdiği üç farklı tepki görsel olarak kanıtlanmıştır.

1. %100 İşlemci Kilidi ve Ölçeklendirme (Scale-Out) Reaksiyonu

- Linux sisteminin /dev/zero çekirdek modülü kullanılarak işlemci yapay bir sonsuz döngüye sokulmuş ve %100 oranında kilitlenmiştir.
- CloudWatch bu durumu tespit ettiğinde, ASG politikası tetiklenmiş ve 1 olan sunucu sayısı kriz anında otomatik olarak 3'e çıkarılmıştır (*Launching a new EC2 instance*).

2. Otomatik Onarım (Self-Healing) Mekanizması

- Kurulumlar sırasında bağımlılık eksikliği olan veya Load Balancer'ın Sağlık Kontrolünden (Health Check) geçemeyen sunucular tespit edilmiştir.
- AWS, yanıt vermeyen makinenin fişini çekmiş (*Terminating EC2 instance in response to an ELB system health check failure*) ve anında yerine sağlam, yeni bir sunucu açarak sistemin kendi kendini onardığını kanıtlamıştır.

3. Kriz Sonrası Küçülme (Scale-In) ve Maliyet Tasarrufu

- Test bitirilip işlemci üzerindeki yük kaldırıldığında ortalama CPU kullanımı %0'a inmiştir.
- Sistem, gereksiz kaynak israfını önlemek amacıyla yeni açtığı sunucuları sırasıyla imha etmiş ve kapasiteyi başlangıç seviyesi olan 1'e düşürerek (*shrinking the capacity from 3 to 1*) uyku durumuna geri dönmüştür.

SONUÇ VE DEĞERLENDİRME

Bu proje kapsamında; sadece bir web uygulamasının ayağa kaldırılmasından ziyade, AWS servisleri (EC2, ASG, ALB, CloudWatch) entegre edilerek endüstriyel standartlarda, çökmeye karşı dayanıklı ve elastik bir bulut bilişim altyapısı inşa edilmiştir. Stres altındaki büyüme, donanımsal arıza anındaki onarım ve kriz sonrasındaki küçülme testleri başarıyla gerçekleştirilerek tam otomatize edilmiş bir mimari elde edilmiştir.

CPU Stres Testi (Siber Yük Simülasyonu):

Auto Scaling sisteminin çalışıp çalışmadığını kanıtlamak için CloudWatch'un ölçüm yaptığı CPU değerini kasten %10'un (Hedef değer) üzerine çıkarmamız gerekiyordu. Web arayüzünden sürekli tıklamak yerine, Linux'un kendi çekirdek mimarisini kullanarak işlemciyi "Sonsuz bir Kısır Döngüye" soktuk. Bu sayede sunucu anında %100 CPU kapasitesine kilitlendi ve AWS alarm vererek yeni sunucuları (Scale-Out) başarıyla açmak zorunda kaldı. Test bitiminde de sistemi normale döndürmek için bu süreci iptal eden kapatma komutunu kullandık.

İşlemciyi %100'e Kilitleme Komutu:

```
cat /dev/zero > /dev/null &
```

Krizi Bitirme ve İşlemciyi Rahatlatma Komutu:

```
killall cat
```

🔥 EFSANE CUMA İNDİRLİMLERİ 🔥

Bulut Mimari Kitabı %80 İndirimle!

Hemen Satın Al

Activity history (4)

🔍 Filter activity history

📅 Filter by a date and time range

< 1 > ⚙️

Status	Description	Cause	Start time	E
⌚ Not yet in service	Launching a new EC2 instance: i-01ee0024276435378	At 2026-06-15T09:17:14Z a monitor alarm TargetTracking-Eticaret-ASG-AlarmHigh-477dcd5e-c832-40ce-ab7c-dd2f75f5be9cd in state ALARM triggered policy Target Tracking Policy changing the desired capacity from 1 to 3. At 2026-06-15T09:17:21Z an instance was started in response to a difference between desired and actual capacity, increasing the capacity from 1 to 3.	2026 June 15, 12:17:23 PM +03:00	
⌚ Not yet in service	Launching a new EC2 instance: i-0ef9232d074c8e633	At 2026-06-15T09:17:14Z a monitor alarm TargetTracking-Eticaret-ASG-AlarmHigh-477dcd5e-c832-40ce-ab7c-dd2f75f5be9cd in state ALARM triggered policy Target Tracking Policy changing the desired capacity from 1 to 3. At 2026-06-15T09:17:21Z an instance was started in response to a difference between desired and actual capacity, increasing the capacity from 1 to 3.	2026 June 15, 12:17:23 PM +03:00	
✅ Successful	Launching a new EC2 instance: i-068d9aab723c5507d	At 2026-06-14T15:47:19Z a user request created an AutoScalingGroup changing the desired capacity from 0 to 1. At 2026-06-14T15:50:21Z an instance was started in response to a difference between desired and actual capacity, increasing the capacity from 0 to 1.	2026 June 14, 06:50:23 PM +03:00	2 1 P
✅ Successful	Updating load balancers/target groups: Successful. Status Reason: Added: arn:aws:elasticloadbalancing:us-east-		2026 June 14, 06:47:33 PM +03:00	2 1 P

Stres Testi Altında Otomatik Büyüme (Scale-Out) Reaksiyonu

İşlemci yükünün kasıtlı olarak kilitlenmesiyle CloudWatch alarminin tetiklendiği an. Auto Scaling mekanizmasının "Target Tracking" politikası gereği, olası bir sistem çökmesini engellemek için kapasiteyi anında 1'den 3'e çıkararak yeni yedek sunucuların kurulumunu (Not yet in service) başlattığı görülmektedir.

Activity history (6)					
Filter activity history			Filter by a date and time range		
Status	Description	Cause	Start time		
Successful	Launching a new EC2 instance: i-0cf8803e776eaa632	At 2026-06-15T09:22:59Z an instance was launched in response to an unhealthy instance needing to be replaced.	2026 June 15, 12:23:00 PM +03:00	21	1!
Successful	Terminating EC2 instance: i-0ef9232d074c8e633	At 2026-06-15T09:22:59Z an instance was taken out of service in response to an ELB system health check failure.	2026 June 15, 12:22:59 PM +03:00	21	1!
Successful	Launching a new EC2 instance: i-01ee0024276435378	At 2026-06-15T09:17:14Z a monitor alarm TargetTracking-Eticaret-ASG-AlarmHigh-477dcd5e-c832-40ce-ab7c-dd2f75f9e9cd in state ALARM triggered policy Target Tracking Policy changing the desired capacity from 1 to 3. At 2026-06-15T09:17:21Z an instance was started in response to a difference between desired and actual capacity, increasing the capacity from 1 to 3.	2026 June 15, 12:17:23 PM +03:00	21	1!
Successful	Launching a new EC2 instance: i-0ef9232d074c8e633	At 2026-06-15T09:17:14Z a monitor alarm TargetTracking-Eticaret-ASG-AlarmHigh-477dcd5e-c832-40ce-ab7c-dd2f75f9e9cd in state ALARM triggered policy Target Tracking Policy changing the desired capacity from 1 to 3. At 2026-06-15T09:17:21Z an instance was started in response to a difference between desired and actual capacity, increasing the capacity from 1 to 3.	2026 June 15, 12:17:23 PM +03:00	21	1!
Successful	Launching a new EC2 instance: i-068d9aab723c5507d	At 2026-06-14T15:47:19Z a user request created an AutoScalingGroup changing the desired capacity from 0 to 1. At 2026-06-14T15:50:21Z an instance was started in response to a difference between desired and actual capacity, increasing the capacity from 0 to 1.	2026 June 14, 06:50:23 PM +03:00	21	1!

Sistemin Kendi Kendini Onarması (Self-Healing) ve Başarılı Ölçeklendirme Kriz anında açılan sunuculardan birinin Load Balancer sağlık kontrolünden (Health Check) geçememesi üzerine, AWS'nin arızalı makineyi otomatik olarak imha edip (Terminating) yerine anında yeni ve sağlıklı bir sunucu (Launching) açarak mimariyi insan müdahalesi olmadan onardığı aktivite geçmişti.

```
from flask import Flask, render_template
import math

app = Flask(__name__)

@app.route('/')
def index():
    return render_template('index.html')

# Bu rota, Auto Scaling'i tetiklemek için CPU'yu kasten yoracak
@app.route('/satin-al')
def satin_al():
    sonuc = 0
    # İşlemciyi terleten o meşhur stres döngüsü
    for i in range(1, 15000000):
        sonuc += math.sqrt(i)

    return f"<h1>Tebrikler! Siparişiniz Alındı.</h1><p>İşlemci yoruldu, hesaplanan değer: {sonuc}</p>"

if __name__ == '__main__':
    # Bulut sunucusunda tüm dış bağlantılara açmak için host='0.0.0.0' yapıyoruz
    app.run(host='0.0.0.0', port=5000, debug=True)
```


AWS Auto Scaling mekanizmasının (otomatik büyüme ve küçülme) çalışabilmesi için sistemin "krize girdiğine" ikna olması gerekmektedir. Gerçek dünyadaki "Efsane Cuma" trafiğini simüle edebilmek için, sunucuya gelen istekleri sadece yanıtlamakla kalmayıp donanımı sonuna kadar zorlayacak matematiksel bir yük kodlanmıştır. Bu kod, Load Balancer üzerinden gelen trafiği karşılayan asıl e-ticaret arka plan (backend) uygulamasıdır.

Satır Satır İşlevleri:

- **@app.route('/') (Ana Sayfa Rotası):** Kullanıcı Load Balancer'ın adresine girdiğinde onu karşılayan ilk duraktır. Sadece görsel bir arayüz olan index.html sayfasını render ederek müşteriye gösterir. Bu aşamada sunucu işlemcisi rahattır.
- **@app.route('/satin-al') (Stres Tetikleyici Rota):** Müşteri e-ticaret sayfasındaki "Hemen Satın Al" butonuna bastığı an bu fonksiyon tetiklenir. Auto Scaling mekanizmasının kalbi burasıdır.
- **for i in range(1, 15000000): sonuc += math.sqrt(i):** Sistemin kasten kilitlendiği ana simülasyon döngüsüdür. Gelen her bir satın alma isteği için işlemciye tam 15 milyon kez karekök (math.sqrt) alma işlemi yaptırılır. Bu ağır matematiksel döngü, t2.micro gibi küçük bir sunucunun CPU kapasitesini saniyeler içinde %100'e çıkararak CloudWatch alarmlarını tetikler.
- **app.run(host='0.0.0.0', port=5000):** Web uygulamasının sadece yerel makineden (localhost) değil, dış dünyadan (AWS Load Balancer üzerinden) gelen tüm isteklere açık olması için Host adresi 0.0.0.0 olarak tanımlanmıştır. *(Not: Canlı ortam şablonunda bu port, standart web trafiği olan Port 80'e yönlendirilmiştir).*